Java Mock Interview — 400 Questions & Answers

OOPs • Exception Handling • Multithreading • Collection Framework
Short, simple, interview-ready answers for Java fresher/mock interview preparation.

How to use: *Give the direct answer first. If the interviewer asks "Explain", add the supporting point or example. Focus on understanding rather than memorizing.*

PART 1 — OOPs

Object-Oriented Programming

1. What is OOP?
Answer: OOP stands for Object-Oriented Programming. It is a programming approach based on classes and objects.

2. What is a class?
Answer: A class is a blueprint or template used to create objects.

3. What is an object?
Answer: An object is an instance of a class.

4. What does an object contain?
Answer: An object has state represented by fields and behavior represented by methods.

5. What are the four pillars of OOP?
Answer: Encapsulation, Inheritance, Polymorphism, and Abstraction.

6. What is encapsulation?
Answer: Encapsulation means wrapping data and methods together and controlling access to the data.

7. How do you achieve encapsulation?
Answer: Usually by making fields private and accessing them through methods such as getters and setters.

8. What is inheritance?
Answer: Inheritance allows a child class to acquire accessible members of a parent class.

9. Which keyword is used for class inheritance?
Answer: The extends keyword is used.

10. Why do we use inheritance?
Answer: Mainly for code reuse and to represent an IS-A relationship.

11. What is polymorphism?
Answer: Polymorphism means one thing can have multiple forms.

12. What are the types of polymorphism in Java?
Answer: Compile-time polymorphism and runtime polymorphism.

13. What is compile-time polymorphism?
Answer: Method overloading is the common example of compile-time polymorphism.

14. What is runtime polymorphism?
Answer: Method overriding is the common example of runtime polymorphism.

15. What is abstraction?
Answer: Abstraction means hiding implementation details and showing essential functionality.

16. How is abstraction achieved in Java?
Answer: Using abstract classes and interfaces.

17. What is an IS-A relationship?
Answer: It represents inheritance; for example, a Dog IS-A Animal.

18. What is a HAS-A relationship?

Answer: It represents a composition or aggregation relationship; for example, a Car HAS-A Engine.

19. What is association?

Answer: Association is a relationship between two objects that can exist independently.

20. What is aggregation?

Answer: Aggregation is a weak HAS-A relationship where the contained object can exist independently.

21. What is composition?

Answer: Composition is a strong HAS-A relationship where the contained object's lifecycle is tied to the owner.

22. Composition vs inheritance?

Answer: Composition represents HAS-A, while inheritance represents IS-A.

23. Why is composition often preferred over inheritance?

Answer: It usually provides more flexibility and reduces tight coupling.

24. What is tight coupling?

Answer: Tight coupling means one class strongly depends on another specific class.

25. What is loose coupling?

Answer: Loose coupling means classes depend less on specific implementations, often through abstractions.

26. What is a constructor?

Answer: A constructor initializes an object when the object is created.

27. Does a constructor have a return type?

Answer: No.

28. Can constructors be overloaded?

Answer: Yes.

29. Can constructors be overridden?

Answer: No.

30. Can constructors be inherited?

Answer: No.

31. What is a default constructor?

Answer: If no constructor is declared, the compiler provides a no-argument constructor.

32. What happens if you define a parameterized constructor?

Answer: The compiler does not automatically provide the default no-argument constructor.

33. Can a constructor be private?

Answer: Yes.

34. Why use a private constructor?

Answer: It can restrict object creation, such as in Singleton-style designs.

35. What is this?

Answer: this refers to the current object.

36. Why is this used?

Answer: It is commonly used to refer to current instance members and distinguish fields from parameters.

37. Can this call another constructor?

Answer: Yes, using this().

38. What is super?

Answer: super refers to the immediate parent-class part of the current object.

39. Why is super used?

Answer: It can access parent members and call the parent constructor.

40. What is super()?
Answer: It calls a parent-class constructor.

41. Where must this() appear?
Answer: If used, it must be the first statement of the constructor.

42. Where must super() appear?
Answer: If explicitly used, it must be the first statement of the constructor.

43. Can this() and super() be used together in one constructor?
Answer: No, because both must be the first statement.

44. What happens if super() is not written?
Answer: The compiler tries to insert a call to the parent no-argument constructor.

45. Can an abstract class have a constructor?
Answer: Yes.

46. What is method overloading?
Answer: It means using the same method name with different parameter lists.

47. Can methods be overloaded by changing only return type?
Answer: No.

48. Can overloaded methods have different access modifiers?
Answer: Yes, if their parameter lists are different.

49. Can static methods be overloaded?
Answer: Yes.

50. Can constructors be overloaded?
Answer: Yes.

51. What is method overriding?
Answer: It is when a child class provides its own implementation of an inherited method.

52. Which annotation is commonly used for overriding?
Answer: @Override.

53. Can static methods be overridden?
Answer: No. Static methods are hidden, not overridden.

54. Can private methods be overridden?
Answer: No, because private methods are not inherited.

55. Can final methods be overridden?
Answer: No.

56. Can an overriding method have a more restrictive access modifier?
Answer: No.

57. Can an overriding method have a wider access modifier?
Answer: Yes.

58. What is a covariant return type?
Answer: An overriding method can return a subtype of the original method's return type.

59. Can final methods be overloaded?
Answer: Yes. final prevents overriding, not overloading.

60. Can private methods be overloaded?
Answer: Yes.

61. Can static methods be overloaded?
Answer: Yes.

62. What is dynamic method dispatch?

Answer: It is runtime selection of an overridden method based on the actual object.

63. Give an example of runtime polymorphism.

Answer: Animal a = new Dog(); a.sound(); can call Dog's overridden sound method at runtime.

64. Overloading vs overriding?

Answer: Overloading uses different parameters and is resolved at compile time; overriding uses the same signature and is resolved at runtime.

65. What is an abstract class?

Answer: A class declared abstract that can contain abstract and concrete methods.

66. Can we create an object of an abstract class?

Answer: No.

67. Can an abstract class have constructors?

Answer: Yes.

68. Can an abstract class have variables?

Answer: Yes.

69. Can an abstract class have static methods?

Answer: Yes.

70. Can an abstract class have final methods?

Answer: Yes.

71. What is an abstract method?

Answer: A method declared without an implementation.

72. Can an abstract class have no abstract methods?

Answer: Yes.

73. What is an interface?

Answer: An interface defines a contract that implementing classes follow.

74. Which keyword implements an interface?

Answer: implements.

75. Can a class implement multiple interfaces?

Answer: Yes.

76. Why can Java implement multiple interfaces?

Answer: A class can implement multiple contracts without extending multiple classes.

77. Can an interface have variables?

Answer: Yes; interface fields are implicitly public, static, and final.

78. Can an interface have default methods?

Answer: Yes.

79. Can an interface have static methods?

Answer: Yes.

80. Can an interface have private methods?

Answer: Yes, modern Java supports private interface methods.

81. Can we create an interface object?

Answer: No, but an interface reference can refer to an implementing object.

82. Abstract class vs interface?

Answer: An abstract class can have instance state and constructors; an interface mainly defines a contract and supports multiple implementation.

83. Can an interface extend another interface?

Answer: Yes.

84. Can an interface extend multiple interfaces?
Answer: Yes.

85. What is static?
Answer: static makes a member belong to the class rather than to each object.

86. What is a static variable?
Answer: A class-level variable shared by objects of that class.

87. What is a static method?
Answer: A method associated with the class rather than a particular object.

88. Can a static method directly access an instance variable?
Answer: No, because an instance variable requires an object.

89. Why can't a static method directly access an instance variable?
Answer: A static method can run without any object, but an instance variable belongs to an object.

90. What is a final variable?
Answer: A variable that cannot be reassigned after initialization.

91. What is a final method?
Answer: A method that cannot be overridden.

92. What is a final class?
Answer: A class that cannot be extended.

93. Can a final reference object's state be modified?
Answer: Yes, the reference cannot be reassigned, but a mutable object may still change.

94. Does Java support multiple inheritance with classes?
Answer: No.

95. Why not?
Answer: It avoids ambiguity such as the diamond problem.

96. How can Java achieve multiple inheritance of type?
Answer: Through interfaces.

97. What is upcasting?
Answer: Assigning a child object to a parent reference, such as Animal a = new Dog().

98. What is downcasting?
Answer: Casting a parent reference to a child type when the actual object is compatible.

99. What is the most important OOP concept for runtime behavior?
Answer: Runtime polymorphism through method overriding and dynamic method dispatch.

100. What is the difference between a class and an object?
Answer: A class is a blueprint; an object is a real instance created from that class.

PART 2 — Exception Handling

Exception Handling

1. What is an exception?
Answer: An exception is an event that disrupts the normal flow of program execution.

2. What is exception handling?
Answer: It is a mechanism for handling exceptional conditions in a controlled way.

3. Which keywords are used for exception handling?
Answer: try, catch, finally, throw, and throws.

4. What is try?
Answer: try contains code that may throw an exception.

5. What is catch?
Answer: catch handles a matching exception from the try block.

6. What is finally?
Answer: finally contains cleanup code that normally executes whether an exception occurs or not.

7. What is throw?
Answer: throw explicitly throws an exception.

8. What is throws?
Answer: throws declares exceptions that a method may pass to its caller.

9. What is Throwable?
Answer: Throwable is the root type for Java exceptions and errors.

10. What are the main children of Throwable?
Answer: Exception and Error.

11. What is Error?
Answer: Error represents serious problems that applications generally should not recover from.

12. Give examples of Error.
Answer: OutOfMemoryError and StackOverflowError.

13. What is Exception?
Answer: Exception represents conditions that an application may handle.

14. What is RuntimeException?
Answer: RuntimeException is the superclass of many unchecked exceptions.

15. Give an example of RuntimeException.
Answer: NullPointerException is an example.

16. What is a checked exception?
Answer: A checked exception must be handled or declared by the compiler.

17. Give an example of a checked exception.
Answer: IOException is a checked exception.

18. What is an unchecked exception?
Answer: An unchecked exception is generally a RuntimeException or one of its subclasses.

19. Give an example of an unchecked exception.
Answer: NullPointerException is an unchecked exception.

20. Why do checked exceptions exist?
Answer: They force code to explicitly consider certain exceptional conditions.

21. Can Error be caught?
Answer: Yes, technically, but serious Errors are generally not treated as normal recoverable conditions.

22. Can try exist without catch?
Answer: Yes, if it has a finally block.

23. Can catch exist without try?
Answer: No.

24. Can finally exist without try?
Answer: No.

25. Can try have multiple catch blocks?
Answer: Yes.

26. What is the difference between throw and throws?
Answer: throw actually throws an exception; throws declares possible exceptions.

27. Can throw be used for checked exceptions?
Answer: Yes.

28. Can throw be used for unchecked exceptions?
Answer: Yes.

29. Can throws declare multiple exceptions?
Answer: Yes.

30. Does finally normally execute when catch executes?
Answer: Yes.

31. Does finally always execute?
Answer: No; abrupt JVM termination can prevent it.

32. What is exception propagation?
Answer: It is the movement of an exception up the call stack until a suitable handler is found.

33. What happens if an exception is not handled?
Answer: It propagates to the caller; if uncaught in the thread, the thread terminates.

34. Can catch handle a parent exception type?
Answer: Yes.

35. Which catch should come first?
Answer: More specific exception catches should come before broader catches.

36. Why should specific catch blocks come first?
Answer: A broad catch first could make a specific catch unreachable.

37. Can one catch handle multiple exceptions?
Answer: Yes, using multi-catch.

38. Can finally contain return?
Answer: Yes, but it is strongly discouraged.

39. Why avoid return in finally?
Answer: It can suppress an earlier return or exception.

40. Can finally modify a returned object?
Answer: Yes, if the returned object is mutable.

41. What is try-with-resources?
Answer: It automatically closes resources that implement AutoCloseable.

42. Why use try-with-resources?
Answer: It makes resource cleanup automatic and safer.

43. What are examples of resources?
Answer: Files, streams, and many JDBC resources.

44. What is AutoCloseable?
Answer: An interface for resources that can be automatically closed.

45. Can a custom class be used with try-with-resources?
Answer: Yes, if it implements AutoCloseable.

46. What is a suppressed exception?
Answer: It is an exception from resource closing that is recorded when another exception is already being thrown.

47. Can finally be used for cleanup?
Answer: Yes, but try-with-resources is usually better for AutoCloseable resources.

48. What is a stack trace?
Answer: It shows the sequence of method calls leading to an exception.

49. What does printStackTrace() do?
Answer: It prints exception details and the stack trace.

50. What is an exception message?
Answer: It is descriptive text associated with an exception, available through getMessage().

51. What causes ArithmeticException?
Answer: An illegal arithmetic operation such as integer division by zero.

52. What causes NullPointerException?
Answer: Using a null reference where an object is required.

53. What causes ArrayIndexOutOfBoundsException?
Answer: Accessing an array with an invalid index.

54. What causes StringIndexOutOfBoundsException?
Answer: Accessing an invalid String index.

55. What causes NumberFormatException?
Answer: Trying to convert an invalid string into a number.

56. What is ClassCastException?
Answer: It occurs when an object is cast to an incompatible type.

57. What is IllegalArgumentException?
Answer: It indicates that a method received an inappropriate argument.

58. What is IllegalStateException?
Answer: It indicates a method was called when the object's state does not allow it.

59. What is IOException?
Answer: A checked exception related to input/output operations.

60. What is SQLException?
Answer: A checked exception related to database operations.

61. Is NullPointerException checked?
Answer: No.

62. Is IOException checked?
Answer: Yes.

63. Is SQLException checked?
Answer: Yes.

64. Is ArithmeticException checked?
Answer: No.

65. Is ClassNotFoundException checked?
Answer: Yes.

66. Is ClassCastException checked?
Answer: No.

67. Is NumberFormatException checked?
Answer: No.

68. Is InterruptedException checked?
Answer: Yes.

69. Why is InterruptedException important?
Answer: It indicates a thread was interrupted while waiting, sleeping, or joining.

70. Can RuntimeException be caught?
Answer: Yes.

71. Can an exception be rethrown?
Answer: Yes.

72. Can an exception have another exception as its cause?
Answer: Yes.

73. What is exception chaining?
Answer: It means preserving an original cause while throwing another exception.

74. What does getCause() do?
Answer: It returns the underlying cause of an exception, if one exists.

75. What does initCause() do?
Answer: It can set the cause of an exception when appropriate.

76. What is a custom exception?
Answer: An exception created by the programmer for an application-specific condition.

77. How do you create a checked custom exception?
Answer: Extend Exception.

78. How do you create an unchecked custom exception?
Answer: Extend RuntimeException.

79. Checked custom exception vs unchecked custom exception?
Answer: Exception creates a checked exception; RuntimeException creates an unchecked exception.

80. Can a constructor throw an exception?
Answer: Yes.

81. Can a static block throw a checked exception directly?
Answer: No; the checked exception must be handled inside the static initializer.

82. Can an overriding method throw a broader checked exception?
Answer: No.

83. Can an overriding method throw a narrower checked exception?
Answer: Yes.

84. Can an overriding method throw an unchecked exception?
Answer: Yes.

85. Can an overriding method throw no exception?
Answer: Yes.

86. What happens if an exception occurs in try and no catch matches?
Answer: It propagates after finally, if a finally block exists.

87. Can nested try blocks exist?
Answer: Yes.

88. Can catch contain try-catch?
Answer: Yes.

89. Can finally contain try-catch?
Answer: Yes.

90. Can one try have multiple finally blocks?
Answer: No.

91. Can we manually call finally?
Answer: No; it is a language construct.

92. What is the main goal of exception handling?
Answer: To handle exceptional conditions and keep program behavior controlled.

93. Should we catch Exception everywhere?
Answer: No; catch exceptions where you can meaningfully handle them.

94. Should catch blocks be empty?
Answer: Generally no, because they can hide problems.

95. What is good exception logging?
Answer: Log useful context while preserving the original exception and cause.

96. Why not use exceptions for normal program flow?
Answer: It makes code harder to understand and is not the intended use.

97. What is multi-catch?
Answer: It allows multiple exception types to be handled by one catch block.

98. Difference between Exception and Error?
Answer: Exceptions may often be handled by applications; Errors generally indicate serious JVM or system problems.

99. What is a checked exception's key rule?
Answer: It must be caught or declared.

100. What is an unchecked exception's key rule?
Answer: The compiler does not require it to be caught or declared.

PART 3 — Multithreading

Multithreading

1. What is a thread?
Answer: A thread is a lightweight unit of execution inside a process.

2. What is multithreading?
Answer: Multithreading means running multiple threads concurrently within a process.

3. Why use multithreading?
Answer: To perform tasks concurrently and improve responsiveness and resource utilization.

4. Process vs thread?
Answer: A process is an independent running program; a thread is an execution unit within a process.

5. How can a thread be created?
Answer: By using Thread, Runnable, Callable with executors, or thread builders.

6. Which method contains a thread's task code?
Answer: run().

7. Which method starts a new thread?
Answer: start().

8. start() vs run()?
Answer: start() begins new thread execution; calling run() directly is a normal method call.

9. Can start() be called twice?
Answer: No; it throws IllegalThreadStateException.

10. What is Runnable?
Answer: A functional interface representing a task that returns no result.

11. What is Callable?
Answer: A task interface that can return a result and throw checked exceptions.

12. Runnable vs Callable?
Answer: Runnable returns no result; Callable returns a result.

13. What is thread priority?
Answer: A scheduling hint that may influence execution order, but should not be used for correctness.

14. What is the default thread priority?
Answer: Thread.NORM_PRIORITY, which is 5.

15. What is a daemon thread?
Answer: A background thread that does not by itself keep the JVM alive.

16. How do you make a thread daemon?
Answer: Call setDaemon(true) before starting it.

17. When must setDaemon be called?
Answer: Before the thread is started.

18. What does currentThread() return?
Answer: The currently executing Thread object.

19. What does getName() do?
Answer: It returns the thread's name.

20. What does setName() do?
Answer: It changes the thread's name.

21. What does getId() do?
Answer: It returns the thread's identifier.

22. What does isAlive() do?
Answer: It checks whether a thread has started and has not yet terminated.

23. What does interrupt() do?
Answer: It requests that a thread stop what it is doing or respond to interruption.

24. Does interrupt() forcibly kill a thread?
Answer: No; interruption is cooperative.

25. What are Java thread states?
Answer: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, and TERMINATED.

26. What is sleep()?
Answer: It pauses the current thread for a specified duration.

27. Does sleep() release a lock?
Answer: No.

28. What is join()?
Answer: It makes the current thread wait for another thread to finish.

29. What is yield()?
Answer: It is a scheduler hint that the current thread is willing to yield execution.

30. Does yield() guarantee another thread will run?
Answer: No.

31. What is wait()?
Answer: It makes the current thread wait and releases the object's monitor.

32. What is notify()?
Answer: It wakes one thread waiting on the same object's monitor.

33. What is notifyAll()?
Answer: It wakes all threads waiting on the same object's monitor.

34. Where are wait, notify, and notifyAll defined?
Answer: In Object.

35. Where is sleep defined?
Answer: In Thread.

36. Where must wait() be called?
Answer: While the thread owns the object's monitor, normally inside synchronized code.

37. What happens when wait() is called?
Answer: The thread releases that object's monitor and waits.

38. What happens when notify() is called?
Answer: One waiting thread becomes eligible to continue after it reacquires the monitor.

39. What is synchronized?
Answer: It controls concurrent access to shared resources using intrinsic locking.

40. What is a synchronized method?
Answer: A method whose execution is protected by an object's monitor, or the class monitor if static.

41. What is a synchronized block?
Answer: A block of code protected by a specified monitor.

42. Why use synchronized blocks?
Answer: To protect only the critical section instead of an entire method.

43. What is a monitor lock?
Answer: An intrinsic lock associated with every Java object.

44. Can two threads execute synchronized methods on the same object simultaneously?
Answer: Not when they require the same object's monitor.

45. Can two threads execute synchronized methods on different objects simultaneously?
Answer: Yes, because the monitors are different.

46. What is a static synchronized method?
Answer: It synchronizes using the Class object's monitor.

47. What is a race condition?
Answer: A condition where the result depends on the timing or order of concurrent operations.

48. How can race conditions be prevented?
Answer: Use synchronization, locks, atomic classes, concurrent collections, or immutable designs.

49. What is an atomic operation?
Answer: An operation that appears indivisible to relevant concurrent threads.

50. What is AtomicInteger?
Answer: A class that provides atomic operations on an integer.

51. What is deadlock?
Answer: Deadlock occurs when threads wait indefinitely for resources held by each other.

52. Give a deadlock example.
Answer: Thread 1 holds Lock A and waits for B while Thread 2 holds B and waits for A.

53. How can deadlock be prevented?
Answer: Use consistent lock ordering, avoid unnecessary nested locks, and keep lock scope small.

54. What is starvation?
Answer: A thread waits for a long time because other threads repeatedly get the required resources.

55. What is livelock?
Answer: Threads remain active and respond to each other but make no useful progress.

56. Deadlock vs livelock?
Answer: Deadlocked threads are blocked; livelocked threads keep running but do not progress.

57. What is thread safety?
Answer: Code is thread-safe when it behaves correctly under its intended concurrent access.

58. What is a critical section?
Answer: A section accessing shared mutable state that needs controlled concurrent access.

59. What is mutual exclusion?
Answer: Allowing only one thread at a time into a protected critical section.

60. What is ReentrantLock?
Answer: An explicit lock implementation with features such as tryLock and interruptible locking.

61. ReentrantLock vs synchronized?
Answer: ReentrantLock provides extra control; synchronized is simpler and automatically releases the monitor.

62. What is ReadWriteLock?
Answer: A lock that can allow multiple readers while writes require exclusive access.

63. What is Semaphore?
Answer: A synchronization mechanism that controls access using a number of permits.

64. What is CountDownLatch?
Answer: It lets threads wait until a counter reaches zero.

65. Can CountDownLatch be reset?
Answer: No; it is one-shot.

66. What is CyclicBarrier?
Answer: It makes a fixed group of threads wait for each other at a common barrier.

67. CountDownLatch vs CyclicBarrier?
Answer: Latch is one-shot; a barrier can be reused.

68. What is BlockingQueue?
Answer: A queue that can block when inserting into a full queue or retrieving from an empty queue.

69. Where is BlockingQueue useful?
Answer: Producer-consumer applications.

70. What is the producer-consumer problem?
Answer: Producers create data and consumers process it through a shared buffer.

71. Why use higher-level concurrency utilities?
Answer: They reduce the complexity and risks of manual thread coordination.

72. What is ThreadLocal?
Answer: It provides a separate value for each thread.

73. What is an immutable object?
Answer: An object whose state cannot change after construction.

74. Why are immutable objects useful in multithreading?
Answer: They are safer to share because their state does not change.

75. What is Executor Framework?
Answer: A framework for managing and executing tasks using executors and thread pools.

76. Why use ExecutorService?
Answer: It separates task submission from thread management.

77. What is a thread pool?
Answer: A group of reusable worker threads used to execute tasks.

78. What is a fixed thread pool?
Answer: An executor configured with a fixed number of worker threads.

79. What is a cached thread pool?
Answer: An executor that can create and reuse threads as needed.

80. What is ScheduledExecutorService?
Answer: An executor for delayed and periodic task execution.

81. What is submit()?
Answer: It submits a task and returns a Future.

82. What is execute()?
Answer: It executes a Runnable without returning a Future.

83. execute() vs submit()?
Answer: execute is for direct Runnable execution; submit returns a Future and supports result retrieval.

84. What is Future?
Answer: It represents the result of an asynchronous computation.

85. What does Future.get() do?
Answer: It gets the result and may block until the task finishes.

86. Can Future.get() block?
Answer: Yes.

87. What is shutdown()?
Answer: It stops accepting new tasks while allowing submitted tasks to complete.

88. What is shutdownNow()?

Answer: It attempts to stop running tasks by interruption and returns tasks that never started.

89. Why shut down ExecutorService?

Answer: To release executor resources and allow clean application termination.

90. What is a virtual thread?

Answer: A lightweight JVM-managed thread designed for high concurrency.

91. Which Java version finalized virtual threads?

Answer: Java 21.

92. Platform thread vs virtual thread?

Answer: Platform threads are closely tied to OS threads; virtual threads are lightweight JVM-managed threads.

93. When are virtual threads useful?

Answer: For very large numbers of concurrent, especially I/O-bound, tasks.

94. Should CPU-heavy work automatically use virtual threads?

Answer: No; virtual threads do not create extra CPU capacity.

95. How do you create a virtual-thread-per-task executor?

Answer: Use Executors.newVirtualThreadPerTaskExecutor().

96. Can virtual threads use synchronized?

Answer: Yes.

97. Are virtual threads faster for every application?

Answer: No; their main benefit is high concurrency, especially for blocking I/O.

98. What is concurrency?

Answer: Multiple tasks making progress during overlapping periods.

99. What is parallelism?

Answer: Multiple tasks actually executing at the same time, typically on multiple CPU cores.

100. What is context switching?

Answer: Context switching is the process of saving one thread's execution state and loading another thread's state.

PART 4 — Collection Framework

Collection Framework

1. What is Collection Framework?
Answer: It is a set of interfaces and classes used to store and manipulate groups of objects.

2. What are the main collection interfaces?
Answer: List, Set, Queue, and Map.

3. Is Map a child of Collection?
Answer: No.

4. What is List?
Answer: An ordered collection that allows duplicate elements.

5. What is Set?
Answer: A collection that does not allow duplicate elements.

6. What is Queue?
Answer: A collection generally used to hold elements before processing.

7. What is Map?
Answer: A structure that stores key-value pairs.

8. Give List examples.
Answer: ArrayList, LinkedList, Vector, and Stack.

9. Give Set examples.
Answer: HashSet, LinkedHashSet, and TreeSet.

10. Give Map examples.
Answer: HashMap, LinkedHashMap, TreeMap, Hashtable, and ConcurrentHashMap.

11. What is Iterable?
Answer: An interface that provides the iterator() method for traversal.

12. What is Collection interface?
Answer: The main root interface for List, Set, and Queue-style collections.

13. What is Collections class?
Answer: A utility class containing static methods for working with collections.

14. Collection vs Collections?
Answer: Collection is an interface; Collections is a utility class.

15. Can collections store primitive types directly?
Answer: No; wrapper objects are used.

16. What is autoboxing?
Answer: Automatic conversion from a primitive to its wrapper object.

17. What is unboxing?
Answer: Automatic conversion from a wrapper object to its primitive.

18. Can collections store duplicate values?
Answer: It depends on the collection type.

19. Which collections maintain predictable insertion order?
Answer: List does, and LinkedHashSet/LinkedHashMap do by default.

20. Which collections provide sorted ordering?
Answer: TreeSet sorts elements and TreeMap sorts keys.

21. What is ArrayList?
Answer: A resizable-array implementation of List.

22. Does ArrayList allow duplicates?
Answer: Yes.

23. Does ArrayList allow null?
Answer: Yes.

24. Does ArrayList maintain insertion order?
Answer: Yes.

25. What is ArrayList get complexity?
Answer: O(1) for index access.

26. What is ArrayList contains complexity?
Answer: O(n) in general.

27. What is ArrayList remove(index) complexity?
Answer: Generally O(n) because elements may need shifting.

28. Is ArrayList size fixed?
Answer: No; it grows dynamically.

29. What happens when ArrayList needs more capacity?
Answer: It creates a larger backing array and copies elements.

30. Is ArrayList synchronized?
Answer: No.

31. What is LinkedList?
Answer: A doubly linked list implementation of List and Deque.

32. Does LinkedList allow duplicates?
Answer: Yes.

33. Does LinkedList allow null?
Answer: Yes.

34. What is LinkedList get(index) complexity?
Answer: Generally O(n).

35. What is addFirst() complexity in LinkedList?
Answer: O(1).

36. What is removeFirst() complexity in LinkedList?
Answer: O(1).

37. ArrayList vs LinkedList?
Answer: ArrayList is better for random access; LinkedList is useful for operations at the ends.

38. Is LinkedList always faster for insertion?
Answer: No; finding an arbitrary insertion position can take O(n).

39. Which generally has less per-element memory overhead?
Answer: ArrayList generally has less overhead than LinkedList.

40. Which List is commonly preferred for general use?
Answer: ArrayList, unless the workload specifically benefits from linked/deque behavior.

41. What is Vector?
Answer: A legacy synchronized dynamic-array List implementation.

42. Is Vector synchronized?
Answer: Its legacy methods are synchronized.

43. Vector vs ArrayList?
Answer: Vector is legacy and synchronized; ArrayList is unsynchronized and generally preferred for ordinary use.

44. What is Stack?
Answer: A legacy LIFO class that extends Vector.

45. What is LIFO?
Answer: Last In, First Out.

46. What are Stack methods?
Answer: push(), pop(), peek(), empty(), and search().

47. What does push() do?
Answer: It adds an element to the top of the stack.

48. What does pop() do?
Answer: It removes and returns the top element.

49. What does peek() do?
Answer: It returns the top element without removing it.

50. What is the modern alternative to Stack?
Answer: Deque, commonly ArrayDeque, is generally preferred for new stack implementations.

51. What is Set?
Answer: A collection that does not allow duplicate elements.

52. What is HashSet?
Answer: A hash-based Set implementation.

53. Does HashSet maintain insertion order?
Answer: No guaranteed order.

54. Does HashSet allow null?
Answer: It permits one null element.

55. What is HashSet contains complexity?
Answer: O(1) average under good hash distribution.

56. How does HashSet identify duplicates?
Answer: It uses hashing and equality checks involving hashCode() and equals().

57. What is LinkedHashSet?
Answer: A Set that maintains predictable insertion order.

58. HashSet vs LinkedHashSet?
Answer: Both provide uniqueness; LinkedHashSet also maintains insertion order.

59. What is TreeSet?
Answer: A sorted Set implementation.

60. What structure backs TreeSet?
Answer: It is tree-based and uses a Red-Black Tree through its implementation.

61. What is TreeSet complexity?
Answer: O(log n) for common add, remove, and contains operations.

62. Does TreeSet allow duplicates?
Answer: No.

63. Does TreeSet maintain sorted order?
Answer: Yes.

64. What does first() do in TreeSet?
Answer: It returns the lowest element.

65. What does last() do in TreeSet?
Answer: It returns the highest element.

66. What is Map?
Answer: A key-value data structure.

67. Can a Map have duplicate keys?
Answer: No.

68. Can a Map have duplicate values?
Answer: Yes.

69. What happens when the same key is inserted again?
Answer: The previous value is replaced.

70. What is HashMap?
Answer: A hash-based Map implementation.

71. Does HashMap maintain order?
Answer: No guaranteed iteration order.

72. Does HashMap allow null?
Answer: Yes; it allows one null key and multiple null values.

73. What is HashMap get complexity?
Answer: O(1) average under good hash distribution.

74. What is LinkedHashMap?
Answer: A HashMap-based Map that maintains predictable iteration order.

75. What is the default LinkedHashMap order?
Answer: Insertion order.

76. Can LinkedHashMap maintain access order?
Answer: Yes, when constructed with accessOrder set to true.

77. What is TreeMap?
Answer: A Map that keeps keys sorted.

78. What is TreeMap complexity?
Answer: O(log n) for common put, get, and remove operations.

79. What structure backs TreeMap?
Answer: A Red-Black Tree.

80. What is Hashtable?
Answer: A legacy synchronized Map implementation.

81. Does Hashtable allow null?
Answer: No; it does not allow null keys or null values.

82. What is ConcurrentHashMap?
Answer: A thread-safe Map designed for efficient concurrent access.

83. Does ConcurrentHashMap allow null?
Answer: No.

84. HashMap vs ConcurrentHashMap?
Answer: HashMap is not designed for concurrent modification; ConcurrentHashMap is designed for concurrent access.

85. HashMap vs TreeMap?
Answer: HashMap offers O(1) average lookup without sorted keys; TreeMap keeps keys sorted with O(log n) operations.

86. What is Comparable?
Answer: An interface used to define a class's natural ordering with compareTo().

87. What is Comparator?
Answer: An interface used to define custom ordering with compare().

88. What method does Comparable use?

Answer: compareTo().

89. What method does Comparator use?

Answer: compare().

90. Comparable vs Comparator?

Answer: Comparable defines natural ordering inside the class; Comparator defines custom ordering externally.

91. What is Iterator?

Answer: An interface used to traverse collection elements.

92. What are the main Iterator methods?

Answer: hasNext(), next(), and remove().

93. What does hasNext() do?

Answer: It checks whether another element is available.

94. What does next() do?

Answer: It returns the next element.

95. What does Iterator.remove() do?

Answer: It removes the last element returned by the iterator, when supported.

96. What is ListIterator?

Answer: An iterator specifically for Lists that supports forward and backward traversal and list modifications.

97. Iterator vs ListIterator?

Answer: Iterator supports forward traversal; ListIterator supports forward/backward traversal and additional List operations.

98. What is ConcurrentModificationException?

Answer: It can occur when a collection is structurally modified during fail-fast iteration outside supported iterator operations.

99. What is Collections.sort()?

Answer: A utility method that sorts a List using natural ordering or a Comparator overload.

100. What is the simplest way to remember major collections?

Answer: ArrayList: index access; HashSet: unique; TreeSet: sorted; HashMap: key-value; LinkedHashMap: key-value plus insertion order; TreeMap: sorted keys; ConcurrentHashMap: concurrent access.